PRODUCT REQUIREMENTS DOCUMENT

RITSU Desk

Sistem Informasi Transportasi Mahasiswa

Aplikasi Desktop Rich UI berbasis WPF dan C# yang memakai data RITSU lewat RITSU Desktop API

Informasi Dokumen	Deskripsi
Nama Produk	RITSU Desk (Sistem Informasi Transportasi Mahasiswa)
Jenis Dokumen	Product Requirements Document (PRD)
Versi	2.2 (Disesuaikan dengan implementasi RITSU Desk 1.0; diadaptasi dari PRD SISMA 1.0)
Platform	Desktop Windows (WPF), Windows 10 1903+ atau Windows 11
Teknologi Utama	C#, XAML, .NET 10 (`net10.0-windows`), HttpClient, System.Text.Json, xUnit
Sistem Induk	RITSU (Rute ITS Utama), github.com/valdi-at/kewirausahaan-berbasis-teknologi-28-ritsu
Antarmuka Data	RITSU Desktop API v1 (`/api/desktop/v1`) yang ditambahkan ke aplikasi web RITSU
Database	Database PostgreSQL RITSU yang sama dengan aplikasi web, diakses hanya oleh server web
Referensi Desain	Tata letak dari desain SISMA; warna dan gaya visual dari tema RITSU web (`app/globals.css`)
Dokumen Terkait	`PROMPT.md` (perubahan repo RITSU, lihat Lampiran C), `README.md` (build, konfigurasi, pengujian)

Dokumen kebutuhan produk untuk acuan desain, implementasi, dan pengujian.

Daftar Isi

1. Ringkasan Eksekutif 3
2. Latar Belakang dan Permasalahan 3
3. Sasaran, Indikator, dan Batasan 3
4. Pengguna dan Hak Akses 4
5. Ruang Lingkup Produk 4
6. Struktur Navigasi dan Halaman 5
7. Kebutuhan Fungsional 6
8. User Stories dan Kriteria Penerimaan 7
9. Kebutuhan UI/UX dan Design System 8
10. Kebutuhan Nonfungsional 9
11. Rancangan Data (Database RITSU) 9
12. Integrasi dengan RITSU Desktop API 11
13. Rekomendasi Arsitektur Teknis 13
14. Alur Pengguna Utama 14
15. Rencana Implementasi Bertahap 14
16. Strategi Pengujian 15
17. Risiko dan Mitigasi 16
18. Asumsi dan Keputusan yang Masih Dibutuhkan 16
19. Definisi Selesai (Definition of Done) 17
Lampiran A: Checklist Persetujuan PRD 17
Lampiran B: Definisi Metrik Dashboard 17
Lampiran C: Perubahan di Repo RITSU 18
Lampiran D: Status Implementasi RITSU Desk 1.0 18

1. Ringkasan Eksekutif

RITSU (Rute ITS Utama) adalah aplikasi web ojek kampus ITS. Mahasiswa memesan driver, melacak perjalanan, chat dengan driver, dan membayar lewat aplikasi itu. Driver RITSU juga mahasiswa, yang mendaftar lewat pengajuan driver dan disetujui admin.

RITSU Desk adalah aplikasi desktop sistem informasi untuk data RITSU tersebut. Aplikasi ini tidak menggantikan alur pemesanan di web. Fokusnya ada di sisi informasi: mahasiswa penumpang melihat riwayat perjalanan dan pengeluaran, mahasiswa driver melihat perjalanan dan pendapatan, dan admin memantau operasional, meninjau pengajuan driver, mengelola pengguna, serta membaca laporan keuangan.

Data RITSU Desk berasal dari database PostgreSQL yang sama dengan RITSU web, tetapi desktop tidak terhubung langsung ke database. Desktop memanggil RITSU Desktop API, kumpulan endpoint HTTP baru di aplikasi web RITSU. Server web yang menjalankan query, memeriksa hak akses, dan menerapkan aturan bisnis. Akun untuk login sama dengan akun RITSU web, dan tidak ada database terpisah, salinan data, atau proses sinkronisasi.

1.1 Tujuan Produk

- Menyediakan aplikasi desktop yang menampilkan data RITSU dari database yang dipakai aplikasi web, lewat API yang aman.
- Memberi mahasiswa (penumpang dan driver) ringkasan aktivitas mereka sendiri di satu tempat.
- Mempercepat pekerjaan admin seperti peninjauan pengajuan driver, pengaturan peran pengguna, dan pemantauan pemesanan.
- Menyajikan statistik dan laporan keuangan yang dihitung oleh query yang sama dengan dashboard admin web, supaya kedua aplikasi tidak menampilkan angka berbeda.
- Tidak menyimpan kredensial database di perangkat pengguna, sehingga aplikasi bisa dipasang di laptop mahasiswa.

1.2 Pernyataan Produk

Untuk mahasiswa ITS yang memakai RITSU dan admin yang mengoperasikannya, RITSU Desk adalah aplikasi desktop yang menampilkan data perjalanan, pembayaran, dan pengajuan driver dari sistem RITSU yang sama, tanpa perlu membuka dashboard web.

1.3 Status

RITSU Desk 1.0 sudah diimplementasikan di repo ini (proyek `RITSU_Desk` dan `RITSU_Desk.Tests`). Dokumen ini sudah disesuaikan dengan aplikasi yang dibangun. Ringkasan status per kebutuhan ada di Lampiran D.

2. Latar Belakang dan Permasalahan

- RITSU web dibangun untuk alur pemesanan berbasis peta. Halaman riwayat web hanya menampilkan pemesanan yang sudah selesai (stage 5), tanpa rekap pengeluaran.
- Driver belum punya rekap pendapatan pribadi. Data pendapatan per driver baru muncul di panel “Top Earning Drivers” pada dashboard admin.
- Dashboard admin web menampilkan banyak angka, tetapi belum ada daftar pemesanan dengan pencarian dan filter, detail per pengguna, atau ekspor laporan.
- Aplikasi desktop yang terhubung langsung ke PostgreSQL harus menyimpan kredensial database di setiap perangkat. Siapa pun yang membaca kredensial itu bisa melewati semua pembatasan peran. Karena itu akses data dipindah ke API di server web.
- Persetujuan pengajuan driver di web menjalankan dua UPDATE terpisah tanpa transaksi, dan penghapusan pengguna yang punya riwayat pemesanan menghasilkan error yang tidak ditangani. Aturan ini perlu dipusatkan di satu service yang dipakai web dan API.

3. Sasaran, Indikator, dan Batasan

Sasaran	Indikator keberhasilan awal
Data bersama	Semua data bisnis berasal dari database RITSU lewat API. Tidak ada salinan data bisnis di perangkat.
Login	Akun yang valid di RITSU web bisa login di RITSU Desk dengan email dan password yang sama.
Informasi mahasiswa	Penumpang melihat riwayat dan pengeluaran sendiri; driver melihat perjalanan dan pendapatan sendiri.

Sasaran	Indikator keberhasilan awal
Dashboard admin	Angka pada kartu statistik sama dengan dashboard admin web karena dihitung oleh modul query yang sama.
Konsistensi data	Setiap perubahan dari RITSU Desk melewati service yang sama dengan web dan tampil benar saat dibuka di web.
Keandalan	Kesalahan jaringan, sesi kedaluwarsa, konflik data, dan data yang masih dipakai ditangani dengan pesan yang jelas.
Keamanan dasar	Tidak ada kredensial database di perangkat; token bearer berumur pendek; otorisasi diperiksa di server.

Batasan utama: RITSU Desk tidak membuat pemesanan, tidak mengubah stage perjalanan, dan tidak menangani chat atau pelacakan lokasi. Fitur itu tetap di web karena membutuhkan peta, GPS, dan interaksi real time antara penumpang dan driver.

Angka seperti 125 mahasiswa, 12 program studi, 8 dosen, dan 3 angkatan pada referensi visual SISMA hanya contoh tata letak. Kartu di RITSU Desk diisi dari respons API.

4. Pengguna dan Hak Akses

Peran mengikuti kolom `users.role` di database RITSU. RITSU Desk tidak menambah peran baru.

Peran (`users.role`)	Siapa	Kebutuhan utama	Hak akses
`customer`	Mahasiswa penumpang	Melihat riwayat perjalanan, pengeluaran, status pembayaran, dan status pengajuan driver.	Endpoint `/me/*` (data milik sendiri).
`driver`	Mahasiswa driver	Melihat perjalanan yang ditangani, pendapatan, dan perjalanan yang belum dibayar.	Endpoint `/me/*` (data milik sendiri).
`admin`	Pengelola RITSU	Memantau operasional, meninjau pengajuan, mengelola pengguna dan metode pembayaran, membaca laporan.	Endpoint `/me/*` dan `/admin/*`; tulis sesuai Bagian 12.5.
Belum login	–	Masuk ke aplikasi.	Hanya `/meta` dan `/auth/login`.

“Data milik sendiri” berarti baris yang `customer_id` atau `driver_id`-nya sama dengan id pengguna di token. Filter ini dijalankan API di sisi server, sehingga desktop tidak bisa meminta data pengguna lain walaupun request-nya dimodifikasi.

Role tidak disimpan di token. API membaca role dari database pada setiap request, jadi perubahan role dari web langsung berlaku di desktop pada request berikutnya. Desktop memakai role dari `/auth/me` hanya untuk menyusun menu.

5. Ruang Lingkup Produk

5.1 In Scope (Rilis Awal)

- Login dengan akun RITSU (email dan password) dan logout.
- Dashboard sesuai peran: statistik platform untuk admin, ringkasan perjalanan untuk penumpang, ringkasan pendapatan untuk driver.
- Riwayat perjalanan pribadi untuk penumpang dan driver, dengan pencarian, filter, dan panel detail.
- Halaman status pengajuan driver untuk mahasiswa.
- Modul admin Pengguna: daftar, pencarian, filter role, detail, ubah role, hapus (dengan pengecekan relasi).
- Modul admin Pengajuan Driver: daftar, filter status, pratinjau foto SIM, setujui, tolak dengan alasan.
- Modul admin Pemesanan: daftar semua pemesanan dengan filter dan detail, hanya baca.
- Laporan keuangan: pendapatan, outstanding, tren bulanan, per metode pembayaran, top driver.
- Ekspor CSV untuk daftar pemesanan dan laporan keuangan.
- Pengaturan alamat server dari layar login, dengan pengecekan `/meta`.
- Navigasi sidebar, dark mode dan light mode.
- Validasi form, konfirmasi aksi berisiko, notifikasi hasil operasi, dan penanganan kesalahan.
- Perubahan di repo RITSU web yang berstatus Wajib pada Lampiran C (RITSU Desktop API v1).

5.2 Out of Scope (Perlu Keputusan atau Rilis Berikutnya)

- Membuat pemesanan, memilih titik di peta, serta menghitung harga dan jarak.
- Mengubah stage perjalanan, konfirmasi driver, tandai lunas, dan pembayaran. Aksi ini terikat ke penumpang atau driver yang bersangkutan dan tetap di web.

- Chat, unggah media, dan pelacakan lokasi driver secara real time.
- Registrasi akun baru dan reset password (tetap lewat web).
- Mengajukan diri sebagai driver, karena butuh unggah foto SIM ke server web.
- Data akademik mahasiswa seperti NIM, departemen, dan angkatan, karena belum ada di database RITSU (lihat Bagian 18).
- Tambah, ubah, dan hapus metode pembayaran, serta periode kustom di laporan keuangan, selama endpoint B5 dan B7 belum tersedia.
- Fitur “ingat saya” dan pencabutan token (B6).
- Ekspor PDF/Excel dan pengiriman laporan lewat email. Ekspor CSV sudah tersedia (FR-22).
- Koneksi langsung dari desktop ke PostgreSQL.

6. Struktur Navigasi dan Halaman

Halaman	Peran	Elemen UI utama	Endpoint utama
Login	Semua	Logo RITSU, email, password, tombol Masuk, tautan daftar ke RITSU web, status server, tautan Ubah alamat server.	`GET /meta`, `POST /auth/login`
Dashboard Saya	customer, driver	Kartu total perjalanan, perjalanan selesai, total pengeluaran atau pendapatan, belum dibayar; grafik per bulan; perjalanan terbaru. Driver bisa memilih data sebagai driver atau penumpang.	`GET /me/summary`, `GET /me/bookings`
Riwayat Perjalanan	customer, driver	Pencarian, filter stage, status bayar, dan periode (preset atau rentang tanggal); DataGrid; panel detail; pagination. Driver bisa memilih sisi driver atau penumpang.	`GET /me/bookings`, `GET /me/bookings/{id}`
Status Driver	customer, driver	Status pengajuan terakhir, tanggal pengajuan dan peninjauan, nomor telepon, alasan penolakan bila ada, tombol buka halaman pengajuan di web.	`GET /me/driver-application`
Profil & Pengaturan	Semua	Username, email, role, tanggal bergabung, waktu sesi berakhir, pilihan tema, alamat API, versi API dan skema, waktu server, zona waktu DB, lokasi file log.	`GET /auth/me`, `GET /meta`
Dashboard Admin	admin	Peringatan pengajuan menunggu; kartu pengguna, pengajuan menunggu, pendapatan, outstanding, volume pemesanan hari/minggu/bulan, rata-rata nilai; grafik pendapatan harian dan per metode; top driver; aktivitas terbaru.	`GET /admin/dashboard`, `GET /admin/activity` (B4)
Pengguna	admin	Pencarian username/email, filter role, DataGrid, panel detail (jumlah pemesanan dan chat, pengajuan terakhir), Ubah Role, Hapus.	`GET/PATCH/DELETE /admin/users`, `GET /admin/users/{id}`
Pengajuan Driver	admin	Filter status, daftar pengajuan, pratinjau foto SIM (klik untuk perbesar), Setujui, Tolak dengan alasan.	`GET /admin/applications`, `.../approve`, `.../reject`, `.../license`
Pemesanan	admin	Pencarian, filter stage, status bayar, metode, dan periode; DataGrid; detail; pagination; ekspor CSV.	`GET /admin/bookings`, `GET /admin/bookings/{id}`, `GET /admin/payment-methods`
Metode Pembayaran	admin	Daftar metode, jenis (tunai/nontunai), jumlah transaksi, info pembayaran. Hanya baca selama B5 belum tersedia.	`GET /admin/payment-methods`
Laporan Keuangan	admin	KPI, tren bulanan, per metode pembayaran, top driver, pemesanan lunas terbaru, ekspor CSV. Periode kustom menunggu B7.	`GET /admin/finance`, `GET /admin/dashboard` (top driver)

Semua path endpoint relatif terhadap `/api/desktop/v1`. Halaman Mahasiswa, Program Studi, Dosen, dan Mata Kuliah dari desain SISMA diganti dengan halaman di atas. Pola tampilannya tetap sama (sidebar, kartu statistik, DataGrid, form, dialog), sedangkan warna dan gaya visual mengikuti RITSU web (Bagian 9).

Menu sidebar per role:

Role	Grup dan item menu
customer, driver	Menu Utama: Dashboard Saya, Riwayat Perjalanan, Status Driver. Akun: Profil & Pengaturan.
admin	Menu Utama: Dashboard, Pengajuan Driver, Pemesanan. Master Data: Pengguna, Metode Pembayaran. Laporan: Laporan Keuangan. Akun: Profil & Pengaturan.

Bagian bawah sidebar berisi tombol ganti tema, nama dan role pengguna, serta tombol logout.

7. Kebutuhan Fungsional

ID	Fitur	Persyaratan	Prioritas
FR-01	Autentikasi	Login lewat `POST /auth/login` dengan email dan password akun RITSU. Pesan gagal sama untuk email tidak terdaftar dan password salah (`INVALID_CREDENTIALS`).	Must
FR-02	Logout	Desktop memanggil `POST /auth/logout`, lalu menghapus token dan data sesi dari memori.	Must
FR-03	Hak akses	Menu disusun dari role di `/auth/me`. Otorisasi sebenarnya ada di API; respons `403` ditampilkan sebagai "akses ditolak".	Must
FR-04	Dashboard admin	Kartu statistik diisi dari `GET /admin/dashboard`, yang memakai modul query yang sama dengan dashboard admin web (Lampiran B).	Must
FR-05	Dashboard mahasiswa	Ringkasan perjalanan dan pengeluaran (customer) atau perjalanan dan pendapatan (driver) dari `GET /me/summary`.	Must
FR-06	Grafik	Admin: pendapatan harian dan pendapatan per metode pembayaran; Laporan Keuangan: tren pendapatan bulanan. Mahasiswa: jumlah perjalanan per bulan (12 bulan terakhir). Grafik digambar dengan kontrol bawaan aplikasi, tanpa library chart; tooltip menampilkan nilai lengkap.	Must
FR-07	Aktivitas terbaru	Feed 15 aktivitas terbaru dari `GET /admin/activity` (B4), diturunkan dari timestamp tabel yang sudah ada. Jika endpoint gagal, hanya panel aktivitas yang menampilkan pesan error.	Should
FR-08	Riwayat perjalanan	Kolom: tanggal, titik jemput, tujuan, jarak, harga, metode pembayaran, stage, status bayar, dan nama driver atau penumpang.	Must
FR-09	Pencarian dan filter	Pencarian teks tidak membedakan huruf besar/kecil. Filter role, stage, status bayar, status pengajuan, dan periode dikirim sebagai query string ke API.	Must
FR-10	Ubah role pengguna	`PATCH /admin/users/{id}` dengan `customer`, `driver`, atau `admin`. Admin tidak bisa mengubah role dirinya sendiri (`SELF_ACTION_FORBIDDEN`).	Must
FR-11	Hapus pengguna	`DELETE /admin/users/{id}` setelah konfirmasi. Pengguna yang punya pemesanan atau pesan chat ditolak (`HAS_DEPENDENCIES`) dan desktop menampilkan jumlah relasinya.	Must
FR-12	Setujui pengajuan driver	`POST /admin/applications/{id}/approve`. Status dan role berubah dalam satu transaksi di server. Pengajuan yang sudah ditinjau ditolak (`ALREADY_REVIEWED`).	Must
FR-13	Tolak pengajuan driver	`POST /admin/applications/{id}/reject` dengan alasan wajib. Status menjadi `declined` dan alasan tersimpan di `admin_message`.	Must
FR-14	Foto SIM	Gambar diambil dari `licenseUrl` (endpoint admin berautentikasi). Jika gagal dimuat, tampil placeholder dan aplikasi tetap berjalan.	Must
FR-15	Daftar pemesanan admin	Semua pemesanan dari `GET /admin/bookings`, hanya baca, dengan pagination di server.	Must
FR-16	Metode pembayaran	Daftar metode, jenis tunai/nontunai, jumlah transaksi, dan info pembayaran (Must). Tambah, ubah, dan hapus dengan aturan `Cash` dilindungi bergantung pada B5 (Should); selama B5 belum tersedia halaman ini hanya baca dan menampilkan keterangannya.	Should
FR-17	Laporan keuangan	Total, bulan ini, tahun ini, outstanding, rata-rata per pemesanan, tren 12 bulan, per metode pembayaran (dengan persentase), top driver, 10 pemesanan lunas terbaru. Top driver diambil dari `GET /admin/dashboard`. Periode kustom bergantung pada B7.	Should
FR-18	Tema UI	Pengguna dapat mengganti dark mode dan light mode dari sidebar atau halaman Profil & Pengaturan. Preferensi disimpan di `%AppData%\RITSU Desk\settings.json`.	Should
FR-19	Notifikasi	Sistem memberi pesan berhasil atau gagal setelah aksi penting.	Must
FR-20	Penanganan error	Kode error API dipetakan ke pesan bahasa Indonesia. Timeout dan server tidak terjangkau ditangani tanpa menutup aplikasi.	Must
FR-21	Muat ulang data	Setiap halaman punya tombol muat ulang dan keterangan waktu terakhir dimuat, karena data juga berubah dari web.	Should
FR-22	Ekspor CSV	Admin dapat mengekspor daftar pemesanan dan laporan keuangan ke CSV. Dibuat di desktop dari data API, tanpa endpoint baru. Ekspor pemesanan mengikuti filter aktif, maksimal 5.000 baris; file UTF-8 dengan BOM agar terbaca benar di Excel.	Could
FR-23	Cek kompatibilitas	Saat dibuka, desktop memanggil `GET /meta` dan menolak login jika `apiVersion` tidak didukung. Pengecekan diulang setelah alamat server diganti, dan sebelum login jika server belum terhubung.	Must
FR-24	Sesi kedaluwarsa	Respons `401` di tengah pemakaian membawa pengguna kembali ke login dengan pesan "Sesi berakhir, silakan login lagi."	Must
FR-25	Alamat server	Alamat server dapat diubah dari layar login (misalnya `10.10.0.1:3000` atau URL lengkap `.../api/desktop/v1`), divalidasi, lalu disimpan di perangkat.	Should

8. User Stories dan Kriteria Penerimaan

US-01: Login dengan akun RITSU

Sebagai mahasiswa yang sudah punya akun RITSU, saya ingin login di aplikasi desktop dengan email dan password yang sama.

- Akun yang baru dibuat di web bisa langsung dipakai login di desktop.
- Email tidak terdaftar dan password salah menghasilkan pesan yang sama: “Email atau password salah.”
- Jika server tidak bisa dihubungi, pesan yang muncul menyebut masalah koneksi, bukan kredensial.
- Jika terlalu banyak percobaan (B3), muncul pesan untuk menunggu sebelum mencoba lagi.

US-02: Melihat riwayat perjalanan (penumpang)

Sebagai mahasiswa penumpang, saya ingin melihat semua perjalanan saya dan berapa yang sudah saya bayar.

- Hanya pemesanan dengan `customer_id` milik saya yang tampil, termasuk saat memakai pencarian dan filter.
- Nama lokasi diambil dari `pickup.name` dan `destination.name` di respons API. Jika kosong, tampil koordinat dengan 4 desimal; jika koordinat juga kosong, tampil teks `raw`.
- Label stage sama dengan yang dipakai web (lihat Bagian 11.3).
- Total pengeluaran diambil dari `paidAmount` pada `/me/summary`.

US-03: Melihat pendapatan (driver)

Sebagai mahasiswa driver, saya ingin melihat perjalanan yang saya tangani dan total pendapatan saya.

- Hanya pemesanan dengan `driver_id` milik saya yang tampil.
- Pendapatan dihitung dari perjalanan yang sudah dibayar. Perjalanan stage 4 atau 5 yang belum dibayar tampil terpisah (`unpaidCount`, `unpaidAmount`).

US-04: Mengecek status pengajuan driver

Sebagai mahasiswa, saya ingin tahu apakah pengajuan driver saya sudah ditinjau.

- Pengajuan terbaru tampil dengan status Menunggu, Disetujui, atau Ditolak.
- Jika ditolak, isi `adminMessage` tampil sebagai alasan.
- Jika belum pernah mengajukan (`application: null`), tampil petunjuk untuk mengajukan lewat RITSU web.

US-05: Meninjau pengajuan driver (admin)

Sebagai admin, saya ingin menyetujui atau menolak pengajuan driver dari desktop.

- Pengajuan berstatus pending tampil paling atas, sama seperti urutan di web.
- Persetujuan mengubah status pengajuan dan role pengguna dalam satu transaksi. Jika salah satu gagal, tidak ada yang berubah.
- Jika pengajuan sudah ditinjau dari web lebih dulu, API mengembalikan `409 ALREADY_REVIEWED`, desktop menampilkan “Data sudah berubah” dan memuat ulang daftar.
- Penolakan tanpa alasan ditolak oleh form dan juga oleh API.
- Hasilnya terlihat di halaman `/driver/application` dan `/admin/dashboard/applications` di web.

US-06: Mengelola pengguna (admin)

Sebagai admin, saya ingin mengubah role atau menghapus akun yang tidak valid.

- Role hanya bisa diubah ke tiga nilai yang valid.
- Pengguna yang punya riwayat pemesanan tidak bisa dihapus; pesan menyebut jumlah pemesanan dan chat yang terkait.
- Admin tidak bisa mengubah role atau menghapus akunnya sendiri, baik dari desktop maupun dari web.
- Perubahan langsung terlihat di RITSU web karena datanya sama.

US-07: Memantau dashboard (admin)

Sebagai admin, saya ingin melihat kondisi platform tanpa membuka web.

- Setiap kartu menampilkan angka yang sama dengan dashboard admin web pada waktu yang sama.
- Jika data kosong, kartu menampilkan nol dan grafik menampilkan empty state.

US-08: Mengganti tema

Sebagai pengguna, saya ingin memilih tema tampilan yang nyaman.

- Pilihan tema diterapkan pada komponen utama.
- Kontras teks dan kontrol tetap terbaca.
- Pilihan tersimpan di perangkat dan dipakai lagi saat aplikasi dibuka.

9. Kebutuhan UI/UX dan Design System

Aspek	Ketentuan desain
Gaya visual	Desktop modern dan profesional, berorientasi produktivitas. Tata letak mengikuti referensi SISMA; warna, bentuk, dan logo mengikuti RITSU web (tema DaisyUI `ritsu` di `app/globals.css`).
Warna	Primary cyan pastel dengan teks gelap untuk aksi utama dan item menu aktif; secondary kuning pastel untuk aksen; latar halaman abu kehijauan; kartu putih. Nilai warna ada di Bagian 9.1.
Layout	Sidebar putih tetap (lebar 224 px) dengan item aktif berbentuk pil cyan; header halaman (judul, subjudul, waktu terakhir dimuat, tombol Muat ulang); area konten berisi kartu statistik, tabel, grafik, dan panel aktivitas; panel detail di sisi kanan tabel.
Tipografi	Segoe UI Variable (bawaan Windows) dengan judul halaman tebal 24 px, judul kartu 15 px, isi 13 px, label kartu statistik huruf kapital 11 px. Font Geist yang dipakai RITSU web belum dibundel.
Bentuk	Kartu radius 20 px dengan bayangan lembut; tombol dan input radius 14 px dan 10 px; chip nilai radius 16 px; badge berbentuk pil.
Komponen	Tombol primary/secondary/danger/ghost/link, input, dropdown, date picker, DataGrid, dialog overlay, badge, toast, toggle tema, pratinjau gambar, logo RITSU (ikon crosshair dalam kotak cyan), kartu statistik dengan chip nilai berwarna, grafik batang.
Badge status	Badge solid berbentuk pil dengan warna sama seperti web, selalu disertai teks label (lihat Bagian 9.1).
Kartu statistik	Label huruf kapital, nilai tebal di dalam chip berwarna lembut sesuai arti: hijau untuk pendapatan, merah untuk outstanding, kuning untuk menunggu atau belum dibayar, cyan untuk jumlah, netral untuk volume. Nilai panjang mengecil agar tidak terpotong.
Format angka	Rupiah dengan format id-ID tanpa desimal (Rp12.000). Jarak dalam km dengan 1 sampai 2 desimal. Waktu dalam WIB. Label grafik memakai bentuk singkat (73,8rb, 1,2jt); nilai lengkap ada di tooltip.
Bahasa	Bahasa Indonesia untuk seluruh label dan pesan, termasuk pesan dari kode error API.
Status UI	Sediakan state loading, sukses, error, data kosong, akses ditolak, server tidak terjangkau, dan sesi berakhir. Error sebelum data pertama dimuat tampil satu halaman dengan tombol Coba lagi; error saat muat ulang tampil sebagai banner di atas data terakhir.
Konsistensi	Margin, padding, radius, ikon, warna, dan ukuran kontrol konsisten lewat resource bersama (`Themes/Colors.Light.xaml`, `Themes/Colors.Dark.xaml`, `Themes/Styles.xaml`).
Aksesibilitas	Teks di atas latar berwarna lembut memakai varian warna yang lebih gelap (kontras minimal 4,5:1), label kontrol jelas, cincin fokus keyboard cyan, status tidak hanya dibedakan dengan warna.
Responsivitas	Ukuran window minimum 1024 × 640 (default 1360 × 860). Tabel dan panel detail tidak saling menutupi; jika ruang tidak cukup, tabel bergulir horizontal.
Tema gelap	RITSU web hanya punya tema terang. Tema gelap desktop memakai aksen cyan dan kuning yang sama di atas permukaan gelap kehijauan.

9.1 Palet warna

Warna tema terang diambil dari tema `ritsu` di RITSU web (nilai OKLCH dikonversi ke hex).

Token	Warna	Pemakaian
Primary	`#00C5C7`	Tombol utama, item menu aktif, logo, grafik, fokus input
Teks di atas primary	`#000809`	Label tombol utama dan item menu aktif
Secondary	`#FFDC75`	Penanda tujuan, tag “ITS Campus Transport”
Latar halaman	`#EBF4F4`	Area konten dan layar login
Permukaan	`#FFFFFF`	Kartu, sidebar, dialog
Teks utama	`#0C1313`	Judul dan isi
Info	`#00ACF3`	Badge stage 2
Success	`#00BF76`	Badge stage 5, Lunas, Disetujui
Warning	`#FDB500`	Badge stage 1, Belum dibayar, Menunggu, role Admin
Error	`#EC305A`	Badge Ditolak, tombol hapus dan tolak

Nilai	Warna badge
Stage 1 Mencari driver	Warning (kuning)
Stage 2 Driver merespons	Info (biru)
Stage 3 dan 4	Primary (cyan)
Stage 5 Selesai	Success (hijau)
Lunas / Belum dibayar	Success / Warning
Menunggu / Disetujui / Ditolak	Warning / Success / Error
Role Admin / Driver / Penumpang	Warning / Primary / netral

10. Kebutuhan Nonfungsional

ID	Kategori	Persyaratan
NFR-01	Performa	Pada perangkat laboratorium yang memenuhi spesifikasi, navigasi dan interaksi UI terasa responsif.
NFR-02	Performa data	Pemanggilan API tidak boleh membuat UI membeku; gunakan async/await. Daftar besar memakai pagination server (`page`, `pageSize` maksimal 100).
NFR-03	Keamanan	HTTPS wajib selain untuk `localhost`, `*.localhost`, dan alamat jaringan lokal (10.x, 172.16–31.x, 192.168.x), karena server pengembangan `10.10.0.1:3000` belum memakai TLS. Token hanya disimpan di memori dan hanya dikirim ke host server yang dikonfigurasi. Tidak ada kredensial database, `SESSION_SECRET`, atau hash password di desktop.
NFR-04	Keandalan	Timeout (15 detik) dan server tidak terjangkau ditangani dengan pesan yang dapat ditindaklanjuti dan opsi coba lagi. Error tak terduga dicatat di log tanpa menutup aplikasi.
NFR-05	Pemeliharaan	Pisahkan View, ViewModel, Model, klien API, dan Service. Klien API diakses lewat interface `IRitsuApiClient` agar bisa di-mock saat pengujian.
NFR-06	Integritas data	Aturan tulis dijalankan di server oleh service bersama (Bagian 12.5). Desktop tidak menduplikasi aturan itu, hanya validasi form untuk kenyamanan.
NFR-07	Kompatibilitas	Windows 10 1903+ atau Windows 11, .NET 10 (LTS); RITSU Desktop API v1.
NFR-08	Logging	Catat error teknis tanpa password, token, isi request, query string, atau header `Authorization`. Log ada di `%LocalAppData%\RITSU Desk\logs\ritsu-desk.log` dan dirotasi setelah 2 MB.
NFR-09	Kompatibilitas API	Desktop hanya bergantung pada field yang terdokumentasi di `docs/desktop-api.md` dan mengabaikan field tambahan, karena perubahan v1 bersifat aditif. Selama dokumen itu belum ada, acuannya `app/lib/api/serialize.ts` di repo RITSU.
NFR-10	Beban server	Tidak ada auto refresh; data dimuat saat halaman pertama kali dibuka dan lewat tombol Muat ulang. Pencarian dikirim 400 ms setelah pengguna berhenti mengetik. Satu `HttpClient` dipakai ulang untuk seluruh aplikasi.
NFR-11	Zona waktu	API mengirim waktu dalam UTC (ISO 8601). Desktop menampilkan dalam WIB. Batas periode statistik dihitung di server (Bagian 12.8).

11. Rancangan Data (Database RITSU)

RITSU Desk tidak punya database sendiri dan tidak mengakses database secara langsung. Bagian ini menjelaskan data di database RITSU yang menjadi sumber respons API, supaya tim desktop memahami arti setiap field. Semua tabel dibuat oleh migrasi `001` sampai `005` di folder `migrations/` repo RITSU, mengikuti kondisi repo pada commit `135b3dc`.

11.1 Tabel sumber

Tabel	Kolom penting	Dipakai untuk	Diakses lewat
`users`	`id, username, email, password_hash, role, driving_license_image, created_at, updated_at`	Login, profil, modul Pengguna	`/auth/*`, `/admin/users*` (`password_hash` tidak pernah dikirim)
`driver_applications`	`id, user_id, phone_number, driving_license_image, status, admin_message, created_at, reviewed_at`	Status driver, modul Pengajuan Driver	`/me/driver-application`, `/admin/applications*`
`bookings`	`id, pickup_location, stops, destination, driver_ready_at, delivery_start_time, delivery_end_time, customer_id, driver_id, payment_method_id, price, distance, payment_status, stage, created_at, updated_at`	Riwayat, dashboard, laporan, modul Pemesanan	`/me/bookings*`, `/me/summary`, `/admin/bookings*`, `/admin/dashboard`, `/admin/finance`
`payment_methods`	`id, name, icon_link, payment_info, created_at`	Label metode, laporan, modul Metode Pembayaran	`/admin/payment-methods`

Tabel	Kolom penting	Dipakai untuk	Diakses lewat
`chat_messages`	`id, booking_id, sender_id, content, content_type, media_url, created_at`	Hitungan relasi sebelum hapus pengguna	`/admin/users/{id}` (`dependencies.chatMessages`)
`beacons`	`driver_id, is_active, latitude, longitude, updated_at`	Tidak dipakai di rilis awal	–
`_migrations`	`name, applied_at`	Versi skema	`/meta` (`schemaVersion`)

11.2 Relasi utama

```
users           1 ──────── * driver_applications   (ON DELETE CASCADE)
users           1 ──────── * bookings.customer_id  (tanpa cascade)
users           1 ──────── * bookings.driver_id    (tanpa cascade, boleh NULL)
users           1 ──────── 0..1 beacons            (ON DELETE CASCADE)
users           1 ──────── * chat_messages         (sender_id, tanpa cascade)
payment_methods 1 ──────── * bookings              (tanpa cascade)
bookings        1 ──────── * chat_messages         (ON DELETE CASCADE)
```

Akibatnya, pengguna yang pernah memesan, pernah menjadi driver, atau pernah mengirim chat tidak bisa dihapus. API mengembalikan `409 HAS_DEPENDENCIES` beserta jumlah relasinya.

11.3 Nilai yang harus sama persis dengan web

Field API (kolom)	Nilai valid	Label di RITSU Desk
`role` (`users.role`)	`customer`, `driver`, `admin`	Penumpang, Driver, Admin
`status` (`driver_applications.status`)	`pending`, `approved`, `declined`	Menunggu, Disetujui, Ditolak
`stage` (`bookings.stage`)	1, 2, 3, 4, 5	1 Mencari driver, 2 Driver merespons, 3 Driver menuju titik jemput, 4 Dalam perjalanan, 5 Selesai
`paymentStatus` (`bookings.payment_status`)	`true`, `false`	Lunas, Belum dibayar
`name` (`payment_methods.name`)	`Cash` punya arti khusus	Web memakai `name != 'Cash'` untuk menentukan metode nontunai

Status penolakan adalah `declined`, bukan `rejected`. Semua nilai ditulis huruf kecil.

11.4 Format data di respons API

- Kunci JSON memakai camelCase. ID berupa string UUID.
- `pickup` dan `destination` berbentuk `{ lat, lng, name, raw }`. Di database keduanya TEXT berisi JSON; API yang mem-parse. `lat`, `lng`, dan `name` bisa `null` jika data lama tidak lengkap.
- `stops` berupa array objek lokasi dengan bentuk yang sama (`{ lat, lng, name, raw }`), saat ini selalu kosong. Desktop juga menerima array string dan mem-parse-nya seperti server.
- `price` (Rupiah) dan `distanceKm` (km) berupa angka JSON. Jumlah (count) berupa integer.
- Semua waktu berupa string ISO 8601 dalam UTC.
- Path file foto SIM tidak dikirim. Pengajuan membawa `licenseUrl` yang menunjuk ke endpoint berautentikasi.

11.5 Pemetaan tipe JSON ke C#

JSON API	C# (System.Text.Json)
string UUID	`Guid`
string	`string`
objek lokasi	`LocationDto` (`lat`, `lng` sebagai `double?`)
array (`items`, `stops`)	`IReadOnlyList<T>`
angka uang dan jarak	`decimal`
integer (count, stage)	`int`
boolean	`bool`
string ISO 8601	`DateTimeOffset`, dikonversi ke WIB untuk tampilan
`null`	tipe nullable (`string?`, `decimal?`, `DateTimeOffset?`)

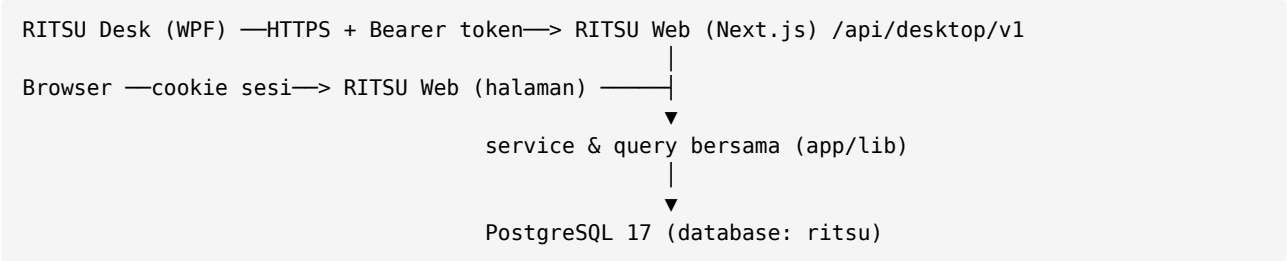
11.6 Aturan perubahan skema dan kontrak

- Desktop tidak pernah menyentuh skema database. Perubahan skema dibuat sebagai migrasi bernomor berikutnya (`006_...sql`) di repo RITSU dan hanya boleh menambah.

- Kontrak desktop adalah API v1, bukan tabel. Selama respons API tidak berubah, skema boleh berubah tanpa memperbarui desktop.
- Perubahan v1 hanya boleh menambah field atau endpoint. Perubahan yang merusak kompatibilitas memakai `/api/desktop/v2`.
- Rilis awal tidak membutuhkan migrasi baru. Migrasi `006` hanya diperlukan jika B6 (pencabutan token) diterapkan.

12. Integrasi dengan RITSU Desktop API

12.1 Arsitektur akses data



Halaman web dan API memanggil service dan modul query yang sama di `app/lib`. Karena itu aturan bisnis dan definisi angka hanya ada di satu tempat.

12.2 Koneksi

- Base URL default diatur di `appsettings.json` (`ApiBaseUrl`), saat ini `http://10.10.0.1:3000/api/desktop/v1` (server pengembangan). Pilihan lain yang tersedia di daftar `KnownServers`: `http://localhost:3000/api/desktop/v1` (RITSU lokal) dan `https://ritsu.shinyshoe.net/api/desktop/v1` (produksi).
- Alamat server bisa diubah dari layar login lewat tautan **Ubah**. Masukan seperti `10.10.0.1:3000` atau `http://localhost:3000` dilengkapi otomatis menjadi `.../api/desktop/v1`. Pilihan pengguna disimpan di `%AppData%\RITSU Desk\settings.json` bersama pilihan tema.
- Untuk pengembangan, jalankan RITSU dari repo-nya dengan `docker compose up -d` (aplikasi dan database), atau `docker compose up -d postgres` lalu `npm run dev`.
- Desktop memakai satu `HttpClient` dengan timeout 15 detik. HTTPS wajib selain untuk `localhost` dan alamat jaringan lokal (NFR-03).
- Saat dibuka, desktop memanggil `GET /meta` dan menampilkan status server di layar login (Terhubung, Tidak terjangkau, Bukan server RITSU, atau versi tidak didukung). Jika `apiVersion` bukan 1, tombol Masuk dinonaktifkan dan muncul pesan versi tidak cocok.

12.3 Autentikasi

- `POST /auth/login` menerima `{ email, password }` dan mengembalikan `{ token, expiresAt, user }`. Server memverifikasi password dengan bcrypt yang sama dengan login web, termasuk trim pada email dan password.
- Token adalah JWT HS256 berumur 12 jam dengan `aud = ritsu-desktop`. Token tidak berisi role. Token web (cookie) tidak diterima API, dan token desktop tidak bisa dipakai sebagai cookie web.
- Desktop menyimpan token di memori dan mengirimnya di header `Authorization: Bearer <token>`. Rilis awal tidak punya fitur “ingat saya”; pengguna login lagi setelah token kedaluwarsa atau aplikasi ditutup.
- `POST /auth/logout` dipanggil saat logout. Tanpa B6 server tidak mencabut token, sehingga token tetap valid sampai kedaluwarsa jika sempat tersalin.
- Desktop tidak membutuhkan `SESSION_SECRET`, kredensial database, atau library bcrypt.

12.4 Daftar endpoint

Semua path relatif terhadap `/api/desktop/v1`.

Method	Path	Peran	Kegunaan
GET	`/meta`	tanpa login	Versi API, versi skema, waktu server, zona waktu database
POST	`/auth/login`	tanpa login	Login, menghasilkan token
GET	`/auth/me`	semua	Data pengguna dan role terkini
POST	`/auth/logout`	semua	Logout

Method	Path	Peran	Kegunaan
GET	`/me/summary?as=`	semua	Ringkasan perjalanan, pengeluaran atau pendapatan, tren bulanan
GET	`/me/bookings`	semua	Riwayat perjalanan sendiri dengan filter dan pagination
GET	`/me/bookings/{id}`	semua	Detail perjalanan sendiri
GET	`/me/driver-application`	semua	Pengajuan driver terakhir
GET	`/admin/dashboard`	admin	Metrik dashboard (Lampiran B)
GET	`/admin/finance`	admin	Metrik laporan keuangan
GET	`/admin/activity`	admin	Aktivitas terbaru (B4)
GET	`/admin/users`	admin	Daftar pengguna
GET	`/admin/users/{id}`	admin	Detail pengguna dan jumlah relasi
PATCH	`/admin/users/{id}`	admin	Ubah role
DELETE	`/admin/users/{id}`	admin	Hapus pengguna
GET	`/admin/applications`	admin	Daftar pengajuan driver
POST	`/admin/applications/{id}/approve`	admin	Setujui pengajuan
POST	`/admin/applications/{id}/reject`	admin	Tolak pengajuan
GET	`/admin/applications/{id}/license`	admin	Foto SIM
GET	`/admin/bookings`	admin	Daftar semua pemesanan
GET	`/admin/bookings/{id}`	admin	Detail pemesanan
GET	`/admin/payment-methods`	admin	Daftar metode pembayaran dan jumlah pemakaian
POST, PATCH, DELETE	`/admin/payment-methods[/{id}]`	admin	Kelola metode pembayaran (B5, belum dipakai desktop)

Parameter filter: `as` (`customer` atau `driver`, default sesuai role), `stage`, `paid`, `paymentMethodId`, `role`, `status`, `q` (pencarian), `from` (inklusif) dan `to` (eksklusif) dalam ISO 8601 terhadap `created_at`, serta `page` dan `pageSize`. Detail lengkap ada di `docs/desktop-api.md` pada repo RITSU.

12.5 Aturan penulisan

Aturan tulis ada di `app/lib/services/admin.ts` pada repo RITSU dan dipakai oleh server action web maupun API. Desktop tidak mengulang aturan ini, hanya menampilkan hasilnya.

Aksi	Endpoint	Aturan di server	Kode error
Ubah role	`PATCH /admin/users/{id}`	Role harus `customer`, `driver`, atau `admin`; tidak boleh mengubah diri sendiri; `updated_at` diperbarui.	`VALIDATION_ERROR`, `SELF_ACTION_FORBIDDEN`, `NOT_FOUND`
Hapus pengguna	`DELETE /admin/users/{id}`	Tidak boleh menghapus diri sendiri; ditolak jika masih punya pemesanan atau chat; pengajuan driver ikut terhapus (cascade).	`SELF_ACTION_FORBIDDEN`, `HAS_DEPENDENCIES`, `NOT_FOUND`
Setujui pengajuan	`POST /admin/applications/{id}/approve`	Hanya dari `pending`; status `approved`, `reviewed_at`, dan role `driver` dalam satu transaksi.	`ALREADY_REVIEWED`, `NOT_FOUND`
Tolak pengajuan	`POST /admin/applications/{id}/reject`	Hanya dari `pending`; alasan wajib dan di-trim; status `declined`, `admin_message`, `reviewed_at`.	`VALIDATION_ERROR`, `ALREADY_REVIEWED`, `NOT_FOUND`
Metode pembayaran	`/admin/payment-methods` (B5)	`Cash` tidak boleh diganti nama atau dihapus; metode yang dipakai tidak boleh dihapus.	`VALIDATION_ERROR`, `HAS_DEPENDENCIES`, `NOT_FOUND`

12.6 Format respons dan kode error

- Daftar berbentuk `{ items, page, pageSize, total }`.
- Error berbentuk `{ "error": { "code", "message", "details" } }`. Desktop memakai `code`, bukan teks `message`, untuk memilih pesan bahasa Indonesia.

HTTP	Kode	Pesan di RITSU Desk
400	`VALIDATION_ERROR`	“Data yang diisi belum valid.” (ditambah detail per field bila ada)
401	`INVALID_CREDENTIALS`	“Email atau password salah.”
401	`UNAUTHORIZED`	“Sesi berakhir, silakan login lagi.” lalu kembali ke halaman login
403	`FORBIDDEN`	“Anda tidak punya akses ke fitur ini.”
403	`SELF_ACTION_FORBIDDEN`	“Anda tidak bisa melakukan aksi ini pada akun sendiri.”

HTTP	Kode	Pesan di RITSU Desk
404	`NOT_FOUND`	“Data tidak ditemukan. Mungkin sudah dihapus.”
409	`ALREADY_REVIEWED`	“Pengajuan ini sudah ditinjau. Daftar dimuat ulang.”
409	`HAS_DEPENDENCIES`	“Pengguna masih punya N pemesanan dan M chat, jadi tidak bisa dihapus.”
429	`RATE_LIMITED`	“Terlalu banyak percobaan login. Coba lagi beberapa menit lagi.”
500	`INTERNAL`	“Terjadi kesalahan di server. Coba lagi nanti.”
–	`NETWORK` (desktop)	“Server tidak dapat dihubungi. Periksa koneksi jaringan dan alamat server, lalu coba lagi.”
–	`TIMEOUT` (desktop)	“Server tidak merespons dalam 15 detik. Coba lagi.”
–	`INVALID_RESPONSE` (desktop)	“Respons server tidak dikenali. Pastikan alamat server mengarah ke RITSU.”

Kode bertanda “(desktop)” dibuat oleh klien saat tidak ada respons API yang bisa dibaca. Untuk `VALIDATION_ERROR`, nama field dari `details.fieldErrors` ditampilkan dalam bahasa Indonesia (misalnya “Periksa: Email, Password.”). Untuk `HAS_DEPENDENCIES`, N adalah `bookingsAsCustomer + bookingsAsDriver` dan M adalah `chatMessages`. Jika body error bukan JSON (misalnya halaman error dari proxy), kode diturunkan dari status HTTP.

12.7 Data yang berubah dari dua sisi

- Web bisa mengubah data kapan saja, jadi tampilan desktop bisa tertinggal. Setiap halaman punya tombol muat ulang dan keterangan waktu terakhir dimuat.
- Konflik tulis ditangani server (misalnya `ALREADY_REVIEWED`). Desktop cukup menampilkan pesan dan memuat ulang.
- Endpoint tulis API memanggil `revalidatePath` yang sama dengan server action web, sehingga halaman admin web ikut memuat data terbaru.

12.8 Zona waktu

- API mengirim semua waktu dalam UTC. Desktop menampilkannya dalam WIB (Asia/Jakarta).
- Angka “hari ini”, “minggu ini”, dan “bulan ini” dihitung server dengan query yang sama dengan web, memakai zona waktu sesi database (kemungkinan besar UTC; nilainya terlihat di `dbTimezone` pada `/meta`). Desktop tidak menghitung ulang batas periode.
- Untuk filter periode di riwayat, desktop mengubah batas hari WIB ke UTC sebelum mengirim `from` dan `to`.

12.9 Foto SIM

- Foto SIM tersimpan di volume `ritsu_uploads` milik container web. Desktop mengambilnya lewat `licenseUrl` dengan header `Authorization` yang sama. `licenseUrl` yang menunjuk ke host lain ditolak, sehingga token tidak pernah terkirim ke luar server yang dikonfigurasi.
- Jika gambar gagal diambil atau tidak bisa dibaca, panel menampilkan placeholder dengan tombol Coba lagi. Klik gambar membuka jendela pratinjau ukuran penuh.
- Dengan B2, route `/uploads/licenses/...` di web juga hanya bisa dibuka admin atau pemilik pengajuan.

13. Rekomendasi Arsitektur Teknis

Lapisan	Tanggung jawab	Contoh
View	Presentasi dan interaksi visual.	`MainWindow.xaml` (overlay dialog dan toast), `LoginView`, `ShellView` (sidebar dan header halaman), `MyDashboardView`, `MyBookingsView`, `DriverStatusView`, `ProfileView`, `AdminDashboardView`, `UsersView`, `ApplicationsView`, `AdminBookingsView`, `PaymentMethodsView`, `FinanceView`; pemetaan ViewModel ke View di `ViewTemplates.xaml`
ViewModel	State UI, command, validasi tampilan, pemanggilan klien API.	`MainViewModel` (login atau shell, sesi berakhir), `ShellViewModel` (menu per role), `LoginViewModel` (termasuk cek `/meta`), `PageViewModel` (dasar: loading, error, muat ulang), `BookingListViewModelBase` (filter, pagination, detail), `UsersViewModel`, `ApplicationsViewModel`, `FinanceViewModel`, dialog di `ViewModels/Dialogs`
Model	DTO yang sesuai dengan respons API.	`UserDto`, `BookingDto`, `LocationDto`, `DriverApplicationDto`, `PagedResult<T>`, `AdminDashboardDto`, `FinanceDto`, `ApiErrorBody`; query `BookingQuery`, `UserQuery`, `ApplicationQuery`
Klien API	Pemanggilan HTTP, serialisasi JSON, pemetaan error, diakses lewat interface.	`IRitsuApiClient`/`RitsuApiClient` (satu interface untuk semua endpoint), `ApiException`, `ServerAddress` (validasi alamat server)
Service	Sesi, pengaturan, tema, pesan error, format tampilan.	`SessionService` (token di memori), `SettingsService`, `ThemeService`, `ErrorMessages`, `Format` (Rupiah, km, WIB), `Labels`, `PeriodFilter`, `Csv`, `DialogService`, `NotificationService`, `PlatformService`

Lapisan	Tanggung jawab	Contoh
Kontrol	Komponen UI bersama.	`StatCard`, `StatusBadge`, `IconChip`, `LogoMark`, `ColumnChart`, `BarList`, `BookingTable`, `BookingFilterBar`, `PagerBar`
Server	Otorisasi, aturan bisnis, query.	RITSU web: `app/api/desktop/v1`, `app/lib/services`, `app/lib/queries`

Teknologi yang dipakai: WPF dengan C# dan XAML di .NET 10, `HttpClient` dengan `System.Net.Http.Json` dan `System.Text.Json`, MVVM tanpa library tambahan (`ObservableObject`, `RelayCommand`, `AsyncRelayCommand`, ObservableCollection), dan pengujian dengan xUnit di proyek `RITSU_Desk.Tests`. Kontrol bawaan memakai tema Fluent WPF (`Application.ThemeMode`, masih berstatus eksperimental dengan peringatan `WPF0001`) yang warna aksen, fokus, dan radiusnya ditimpa lewat resource tema RITSU. Grafik digambar dengan kontrol sendiri sehingga tidak memakai library chart. Desktop tidak memakai Npgsql, Dapper, Entity Framework, atau library bcrypt.

14. Alur Pengguna Utama

14.1 Mahasiswa (penumpang atau driver)

1. Pengguna membuka RITSU Desk. Aplikasi memanggil `/meta` untuk mengecek server dan versi API. Jika server tidak terjangkau, pengguna bisa mengubah alamat server lewat tautan **Ubah**.
2. Pengguna login dengan email dan password akun RITSU.
3. Desktop menyimpan token di memori, membaca role dari respons login, dan membuka Dashboard Saya. Driver bisa beralih antara data sebagai driver dan sebagai penumpang.
4. Pengguna membuka Riwayat Perjalanan, memakai pencarian atau filter, lalu memilih baris untuk membuka panel detail.
5. Pengguna membuka Status Driver untuk melihat hasil pengajuan.
6. Pengguna logout; desktop memanggil `/auth/logout` dan menghapus token.

14.2 Admin

1. Admin membuka RITSU Desk dan login.
2. Dashboard Admin tampil dari `/admin/dashboard`.
3. Admin membuka Pengajuan Driver, memilih pengajuan pending, dan melihat foto SIM.
4. Admin memilih Setujui atau Tolak (dengan alasan).
5. Server mengecek role admin, menjalankan transaksi, dan mengembalikan hasil. Desktop menampilkan notifikasi lalu memuat ulang daftar; panel detail tetap terbuka pada pengajuan yang sama dengan status barunya.
6. Admin membuka Pengguna untuk mencari akun dan mengubah role bila perlu.
7. Admin membuka Pemesanan atau Laporan Keuangan dan bisa mengekspor datanya ke CSV.
8. Admin logout.

15. Rencana Implementasi Bertahap

Fase	Aktivitas	Deliverable	Status RITSU Desk 1.0
Fase 0: API di repo RITSU	Menjalankan `PROMPT.md` di repo RITSU (item Wajib dan item Disarankan yang dipilih), review, merge, deploy.	RITSU Desktop API v1 berjalan di lokal dan produksi.	Di luar repo ini. Desktop sudah terhubung ke API v1 di `10.10.0.1:3000`.
Fase 1: Fondasi	Project WPF, struktur MVVM, design tokens, layout utama.	Shell aplikasi dan navigasi dasar.	Selesai.
Fase 2: Klien API	`RitsuApiClient`, konfigurasi base URL, `/meta`, DTO, pemetaan error, mock untuk pengujian.	Klien API teruji ke RITSU lokal.	Selesai. Diuji dengan stub HTTP; `/meta` dan error tanpa login dicek ke server.
Fase 3: Login dan peran	Login, sesi di memori, menu per role, penanganan `401`.	Login dengan akun RITSU web berfungsi.	Selesai di kode. Login dengan akun nyata belum diuji.
Fase 4: Modul mahasiswa	Dashboard Saya, Riwayat Perjalanan, Status Driver.	Modul penumpang dan driver berfungsi.	Selesai di kode.
Fase 5: Modul admin	Dashboard Admin, Pengguna, Pengajuan Driver, Pemesanan.	Modul admin berfungsi dan cocok dengan web.	Selesai di kode. Kecocokan angka dengan web belum diuji dengan data nyata.
Fase 6: Laporan dan master data	Laporan Keuangan, Metode Pembayaran, ekspor CSV bila disetujui.	Laporan dan master data sesuai prioritas.	Selesai. Metode Pembayaran hanya baca

Fase	Aktivitas	Deliverable	Status RITSU Desk 1.0
			(B5); belum ada periode kustom (B7).
Fase 7: QA lintas aplikasi dan rilis	Tema, penanganan error, uji bersama web, dokumentasi, paket deployment.	Build kandidat rilis dan panduan pengguna.	Sebagian. Tema RITSU, penanganan error, README, dan unit test selesai; uji bersama web dengan akun uji dan paket deployment belum.

Fase 1 dan 2 bisa dikerjakan bersamaan dengan Fase 0 memakai mock klien API. Fase 3 ke atas membutuhkan API yang sudah berjalan di lokal.

16. Strategi Pengujian

Pengujian memakai RITSU yang dijalankan lokal dari repo-nya (`docker compose up -d`) dengan data seed. Aksi tulis tidak diuji ke server produksi. Sisi API diuji dengan skrip smoke test (B8); sisi desktop diuji ke API lokal dan ke mock.

Area	Skenario minimum	Hasil yang diharapkan
Login	Akun dibuat lewat web lalu login di desktop; password salah; email tidak terdaftar; server mati.	Akses sesuai aturan; pesan aman dan jelas.
Token	Token kedaluwarsa; token diubah satu karakter; cookie sesi web dipakai sebagai bearer.	Semua mendapat `401`; desktop kembali ke login.
Isolasi data	Penumpang A meminta `/me/bookings/{id}` milik penumpang B.	`404`; data B tidak pernah tampil.
Hak akses	Akun customer memanggil `/admin/*` langsung (misalnya lewat curl).	`403 FORBIDDEN`.
Dashboard	Bandingkan setiap kartu dengan `/admin/dashboard` web pada data yang sama.	Angka sama.
Pengajuan driver	Setujui di desktop lalu cek web; tolak di desktop lalu cek `/driver/application` di web.	Role dan alasan penolakan tampil benar di web.
Konkurensi	Pengajuan disetujui di web saat desktop masih menampilkan status pending.	Desktop menerima `409 ALREADY_REVIEWED` dan memuat ulang.
Hapus pengguna	Hapus pengguna yang punya pemesanan; hapus pengguna tanpa pemesanan; admin menghapus dirinya sendiri.	`409` dengan jumlah relasi; terhapus; `403`.
Format lokasi	Data lama dengan JSON lengkap, tanpa `name`, dan teks yang bukan JSON.	Tampil nama, koordinat, atau teks mentah tanpa crash.
Zona waktu	Pemesanan pukul 23.30 WIB dan pukul 06.30 WIB; filter periode satu hari.	Tanggal tampil benar dalam WIB; filter memuat pemesanan yang tepat.
Koneksi	Server mati atau jaringan putus saat aplikasi berjalan.	Pesan jelas dan opsi coba lagi; aplikasi tidak crash.
Kompatibilitas	`/meta` mengembalikan `apiVersion` lain.	Desktop menampilkan pesan versi tidak cocok.
Regresi web	Halaman admin web setelah refactor ke service dan query bersama.	Tampilan dan angka sama seperti sebelumnya.
Tema	Pindah dark mode dan light mode.	Komponen utama konsisten dan terbaca.
Usability	Navigasi keyboard, ukuran window, pesan kesalahan.	Pengguna dapat menyelesaikan tugas utama.

16.1 Pengujian otomatis di repo ini

`dotnet test RITSU_Desk.slnx` menjalankan 90 unit test xUnit di proyek `RITSU_Desk.Tests`:

Area	Yang diuji
Klien API	URL dan query string (waktu dalam UTC), header bearer, parsing respons dan body error, `401` mengakhiri sesi tetapi `INVALID_CREDENTIALS` tidak, token kedaluwarsa tidak dikirim, timeout dan gangguan jaringan, `licenseUrl` beda host ditolak, lokasi tanpa nama atau bukan JSON
Alamat server	Normalisasi alamat dan aturan HTTPS
Format dan label	Rupiah, km, waktu WIB (termasuk 23.30 dan 06.30 WIB), label dan warna badge
Filter periode	Batas hari WIB dikonversi ke UTC
Pesan error	Pesan Bagian 12.6, termasuk detail field dan jumlah relasi
CSV	Escape, perlindungan formula spreadsheet, format baris pemesanan
ViewModel	Validasi login dan cek versi API, `ALREADY_REVIEWED` memuat ulang daftar, pesan `HAS_DEPENDENCIES`, aksi ke akun sendiri dinonaktifkan, filter dan pagination, pilihan baris tetap setelah muat ulang

Skenario pada tabel di atas yang membutuhkan akun dan data nyata (login, token, isolasi data, hak akses, kecocokan angka dashboard, aksi tulis, konkurensi) belum dijalankan ke server. RITSU tidak punya akun bawaan: buat akun lewat `/auth/register` di web, lalu jadikan satu akun admin lewat database.

17. Risiko dan Mitigasi

Risiko	Dampak	Mitigasi
Token desktop tersalin	Orang lain bisa memakai akun sampai token kedaluwarsa.	Token 12 jam, hanya di memori, HTTPS wajib. Pencabutan token lewat B6 jika dibutuhkan.
Tebak password lewat API login	Akun dengan password lemah bisa dibobol.	Rate limit dan perbandingan hash palsu (B3).
Refactor merusak halaman admin web	Dashboard atau aksi admin web salah.	Tampilan wajib identik; uji regresi web di Fase 0; review PR sebelum merge.
Push ke `master` langsung deploy	Perubahan belum teruji masuk produksi.	Kerjakan di branch `feat/desktop-api`, review lewat PR, merge setelah uji lokal.
Kontrak API berubah	Desktop gagal membaca respons.	Versi di path (`/v1`), perubahan v1 hanya aditif, `docs/desktop-api.md` sebagai acuan, cek `/meta` saat startup.
Perubahan perilaku web	Admin tidak bisa lagi mengubah role atau menghapus dirinya sendiri dari web.	Disengaja dan dicatat di PR; dikonfirmasi pemilik repo.
Port PostgreSQL terbuka di server	Database bisa diakses dari internet.	B1: tambahkan `ports: !reset []` pada service `postgres` di `docker-compose.prod.yml`. Desktop tidak butuh port ini.
Route `/uploads` tanpa autentikasi	Foto SIM bisa dibuka siapa pun yang tahu path-nya.	B2: route `/uploads/licenses` hanya untuk admin atau pemilik; API memakai endpoint `license` berautentikasi.
Rate limit hilang saat restart	Perlindungan login sementara melemah.	Diterima untuk satu container; pindah ke penyimpanan bersama jika aplikasi diskalakan.
Logika bisnis di code-behind	Kode sulit diuji dan dipelihara.	Pindahkan state dan command ke ViewModel, pemanggilan ke klien API.
Hak akses hanya disembunyikan di UI	Pengguna dapat menjalankan aksi tanpa izin.	Otorisasi selalu di API; menu desktop hanya kenyamanan.
Pengujian ke server produksi	Data asli rusak.	Uji ke RITSU lokal; skrip smoke test menolak URL non-lokal tanpa izin eksplisit.
Library chart tidak kompatibel	Implementasi terlambat.	Sudah dihindari: grafik digambar dengan kontrol sendiri, tanpa library chart.
Tema Fluent WPF masih eksperimental	Tampilan kontrol bawaan bisa berubah di versi .NET berikutnya.	Warna aksen, fokus, dan radius ditimpa lewat resource tema RITSU; periksa ulang tampilan saat upgrade .NET.
HTTP tanpa TLS di jaringan lokal	Token bisa disadap oleh perangkat lain di jaringan yang sama.	HTTP hanya diizinkan untuk localhost dan alamat jaringan lokal; server produksi wajib HTTPS.

18. Asumsi dan Keputusan yang Masih Dibutuhkan

- Versi .NET target dan versi Windows minimum. Diputuskan: .NET 10 (`net10.0-windows`), Windows 10 1903+ atau Windows 11.
- Gaya visual. Diputuskan: warna dan bentuk mengikuti RITSU web (Bagian 9), bukan warna navy dari referensi SISMA.
- Apakah font Geist milik RITSU web perlu dibundel di desktop (lisensi SIL Open Font License). Saat ini memakai Segoe UI Variable.
- Akun uji untuk QA lintas aplikasi. RITSU belum punya akun bawaan; admin pertama harus diset lewat database.
- Item Disarankan mana dari Lampiran C yang masuk rilis awal. Default di `PROMPT.md`: B1, B2, B3, B4, B8.
- Apakah perlu pencabutan token (B6) dan fitur “ingat saya”. Keduanya butuh migrasi `006` dan penyimpanan token yang aman (misalnya DPAPI di Windows).
- Masa berlaku token (usulan 12 jam).
- Zona waktu acuan untuk statistik harian, mingguan, dan bulanan (UTC seperti sekarang, atau WIB). Jika diubah, perubahannya ada di modul query bersama sehingga web dan desktop ikut berubah.
- Apakah perlu data akademik mahasiswa (NIM, departemen, angkatan). Jika perlu, ditambahkan lewat migrasi baru di repo RITSU, misalnya tabel `student_profiles` yang terhubung ke `users.id`, lalu diekspos lewat endpoint baru di v1.
- Apakah aktivitas admin perlu audit log permanen (tabel baru lewat migrasi) atau cukup feed dari timestamp (B4).

- Apakah admin desktop boleh mengubah data pemesanan, misalnya menandai lunas. Rilis awal: tidak; halaman Pemesanan hanya baca.
- Grafik harian web memakai `delivery_end_time`, sedangkan tren bulanan memakai `created_at`. API mengikuti apa adanya. Jika ingin diseragamkan, ubah di modul query bersama.
- Backup, retensi, dan pemulihan mengikuti kebijakan database RITSU (volume `ritsu_pgdata`).

19. Definisi Selesai (Definition of Done)

- Semua item Wajib di Lampiran C sudah di-merge ke repo RITSU, lulus `npm run lint` dan `npm run build`, dan berjalan di lingkungan uji.
- Semua persyaratan berprioritas Must pada rilis yang disetujui telah diimplementasikan di desktop.
- Login berhasil dengan akun yang dibuat di RITSU web.
- Desktop tidak menyimpan kredensial database dan tidak membuka koneksi ke PostgreSQL.
- Setiap aksi tulis lulus uji lintas aplikasi dan hasilnya tampil benar di web.
- Angka dashboard admin cocok dengan dashboard web pada data uji.
- Data mahasiswa dibatasi ke data milik sendiri oleh API, dibuktikan dengan uji isolasi data.
- Password dan token tidak muncul di log desktop maupun server.
- Validasi, pesan error, empty state, server tidak terjangkau, dan sesi berakhir tersedia.
- UI konsisten dengan design system dan mendukung tema yang disepakati.
- Dokumentasi mencakup konfigurasi base URL, cara menjalankan RITSU lokal, tautan ke `docs/desktop-api.md`, dan hasil pengujian (`README.md` di repo ini).

Lampiran A: Checklist Persetujuan PRD

Peran	Nama	Tanggal	Status
Pemilik Produk	_____	_____	Belum disetujui
Dosen/Lead Developer	_____	_____	Belum disetujui
Pemilik Repo dan Database RITSU	_____	_____	Belum disetujui
Perwakilan Pengguna (Mahasiswa)	_____	_____	Belum disetujui

Lampiran B: Definisi Metrik Dashboard

Metrik admin berasal dari query di `app/admin/dashboard/page.tsx` dan `app/admin/dashboard/finance/page.tsx`. Pada Fase 0 query ini dipindah ke `app/lib/queries/admin-dashboard.ts` dan `app/lib/queries/finance.ts` tanpa mengubah definisinya, lalu dipakai oleh halaman web dan API. Metrik mahasiswa ada di `app/lib/queries/me.ts`.

Metrik	Definisi	Endpoint
Pengguna per role	`SELECT role, COUNT(*) FROM users GROUP BY role`	`/admin/dashboard`
Pengajuan per status	`SELECT status, COUNT(*) FROM driver_applications GROUP BY status`	`/admin/dashboard`
Total pendapatan	`SUM(price)` dari `bookings` dengan `payment_status = true`	`/admin/dashboard`
Outstanding	`SUM(price)` dan `COUNT(*)` dengan `payment_status = false AND stage >= 4`	`/admin/dashboard`
Rata-rata nilai pemesanan	`AVG(price)` dengan `payment_status = true`	`/admin/dashboard`
Volume hari ini/minggu/bulan	`COUNT(*)` dengan `created_at >= CURRENT_DATE`, `date_trunc('week', NOW())`, `date_trunc('month', NOW())`	`/admin/dashboard`
Pendapatan harian	`SUM(price)` per `delivery_end_time::date`, 30 hari terakhir, `payment_status = true`, maksimal 14 baris	`/admin/dashboard`
Pendapatan per metode	`SUM(price)` per `payment_methods` (LEFT JOIN), `payment_status = true`	`/admin/dashboard`
Top driver	`SUM(price)` per `driver_id`, `payment_status = true`, 5 teratas	`/admin/dashboard`
Pendapatan bulan ini/tahun ini	`SUM(price)` dengan `payment_status = true` dan `created_at >= date_trunc('month' / 'year', NOW())`	`/admin/finance`
Tren bulanan	`SUM(price)` per `date_trunc('month', created_at)`, 12 bulan terakhir, `payment_status = true`	`/admin/finance`

Metrik	Definisi	Endpoint
Pengeluaran saya	`SUM(price)` dengan `customer_id = :me AND payment_status = true`	`/me/summary?as=customer`
Pendapatan saya	`SUM(price)` dengan `driver_id = :me AND payment_status = true`	`/me/summary?as=driver`
Belum dibayar	`SUM(price)` dan `COUNT(*)` dengan `payment_status = false AND stage >= 4` pada baris milik sendiri	`/me/summary`
Perjalanan saya per bulan	`COUNT(*)` dan `SUM(price)` per `date_trunc('month', created_at)`, 12 bulan terakhir, baris milik sendiri	`/me/summary`

Lampiran C: Perubahan di Repo RITSU

Perubahan ini dijalankan lewat `PROMPT.md`, yang berisi instruksi teknis lengkap untuk Claude Code. ID di bawah sama dengan ID di `PROMPT.md`.

C.1 Wajib

ID	Perubahan	Mendukung
A1	Helper API bersama: format error, respons `no-store`, pagination, serializer (angka, lokasi), dan `withTransaction` di `db.ts`.	Semua endpoint
A2	Token bearer desktop (`aud = ritsu-desktop`, 12 jam, tanpa role di payload) dan `requireApiUser` yang membaca role dari database setiap request.	FR-01 sampai FR-03, FR-24
A3	Service admin bersama di `app/lib/services/admin.ts`, dipakai server action web dan API. Persetujuan jadi transaksional, hapus pengguna menangani relasi, admin tidak bisa mengubah atau menghapus dirinya sendiri, dan UI admin web menampilkan error.	FR-10 sampai FR-13
A4	Modul query bersama untuk dashboard, keuangan, dan data pribadi; halaman admin web memakai modul yang sama tanpa perubahan tampilan.	FR-04 sampai FR-06, FR-17
A5	Endpoint `GET /meta`.	FR-23
A6	Endpoint login, me, logout, dan `/me/*`.	FR-01, FR-02, FR-05, FR-08, US-02 sampai US-04
A7	Endpoint baca admin, termasuk foto SIM berautentikasi dan daftar metode pembayaran.	FR-04, FR-14, FR-15, FR-16 (baca), FR-17
A8	Endpoint tulis admin: ubah role, hapus pengguna, setujui, tolak.	FR-10 sampai FR-13
A9	Dokumentasi `docs/desktop-api.md` dan bagian Desktop API di README.	NFR-09
A10	Verifikasi: lint, build, dan uji manual ke RITSU lokal, termasuk uji isolasi dan regresi halaman admin web.	Bagian 16, 19

C.2 Disarankan

ID	Perubahan	Default di `PROMPT.md`	Mendukung
B1	Tutup port PostgreSQL di `docker-compose.prod.yml`.	Dipilih	Keamanan
B2	Batasi `/uploads/licenses` untuk admin dan pemilik pengajuan.	Dipilih	FR-14, keamanan
B3	Rate limit login desktop dan perbandingan hash palsu untuk email yang tidak terdaftar.	Dipilih	US-01, keamanan
B4	Endpoint `GET /admin/activity` dari timestamp yang sudah ada.	Dipilih	FR-07
B5	Endpoint tulis metode pembayaran dengan perlindungan `Cash`.	Tidak dipilih	FR-16 (tulis)
B6	Pencabutan token lewat tabel `desktop_sessions` (migrasi `006`).	Tidak dipilih	Bagian 12.3, 18
B7	Periode kustom (`from`, `to`) pada `/admin/finance`.	Tidak dipilih	FR-17
B8	Skrip smoke test `scripts/desktop-api-smoke.sh`.	Dipilih	Bagian 16
B9	Spesifikasi OpenAPI 3.1.	Tidak dipilih	NFR-09

Untuk mengubah pilihan, ubah tanda `[x]` atau `[ ]` pada Part B di `PROMPT.md` sebelum menjalankannya, lalu sesuaikan kolom Default di tabel ini.

Lampiran D: Status Implementasi RITSU Desk 1.0

“Selesai” berarti sudah ada di kode dan diperiksa dengan unit test atau render tampilan memakai data contoh. Pengujian dengan akun dan data nyata masih tercatat di Bagian 16.1.

ID	Status	Catatan
FR-01	Selesai	Pesan sama untuk email tidak terdaftar dan password salah; validasi email dan password di form.

ID	Status	Catatan
FR-02	Selesai	Token dihapus walaupun `POST /auth/logout` gagal.
FR-03	Selesai	Menu per role (Bagian 6); `403` tampil sebagai “Anda tidak punya akses ke fitur ini.”
FR-04	Selesai	8 kartu statistik dan peringatan pengajuan menunggu.
FR-05	Selesai	Driver bisa beralih ke data sebagai penumpang.
FR-06	Selesai	Grafik batang dan daftar batang bawaan aplikasi.
FR-07	Selesai	15 aktivitas terbaru.
FR-08	Selesai	Kolom digabung dua baris (Waktu, Rute, Penumpang/Driver, Harga dengan jarak dan metode, Status stage dan bayar) agar tabel muat di samping panel detail.
FR-09	Selesai	Periode berupa preset (hari ini, 7 hari, 30 hari, bulan ini, tahun ini) atau rentang tanggal.
FR-10	Selesai	Dialog pilih role; tombol nonaktif untuk akun sendiri.
FR-11	Selesai	Konfirmasi sebelum hapus; panel detail menampilkan jumlah relasi.
FR-12	Selesai	`ALREADY_REVIEWED` memunculkan peringatan dan memuat ulang daftar.
FR-13	Selesai	Dialog alasan wajib, maksimal 500 karakter.
FR-14	Selesai	Placeholder dan tombol Coba lagi jika gambar gagal; pratinjau ukuran penuh.
FR-15	Selesai	Hanya baca, dengan pagination server.
FR-16	Sebagian	Daftar sudah ada; tambah, ubah, dan hapus menunggu B5.
FR-17	Sebagian	Semua metrik sudah ada; periode kustom menunggu B7.
FR-18	Selesai	Tema terang dan gelap, disimpan di perangkat.
FR-19	Selesai	Toast di kanan bawah; error dan peringatan tampil lebih lama.
FR-20	Selesai	Pesan Bagian 12.6 ditambah pesan koneksi, timeout, dan respons tidak dikenali.
FR-21	Selesai	Tombol Muat ulang dan waktu terakhir dimuat di setiap halaman.
FR-22	Selesai	Ekspor pemesanan (mengikuti filter) dan laporan keuangan.
FR-23	Selesai	Status server di layar login; login dinonaktifkan jika versi API tidak cocok.
FR-24	Selesai	Kembali ke login dengan pesan sesi berakhir.
FR-25	Selesai	Alamat server divalidasi dan disimpan di `%AppData%\RITSU Desk\settings.json`.

Dokumen ini adalah kebutuhan produk berdasarkan referensi desain SISMA, kondisi repo RITSU, dan implementasi RITSU Desk 1.0. Persyaratan yang masih ditandai perlu keputusan harus dikonfirmasi bersama pemilik produk dan pemilik repo RITSU sebelum rilis final.